Which Harness Components Actually Pay? A Switchable-Component Ablation on Local Coding Agents

Anonymous Author(s)
Anonymous Institution

Abstract

The pass rate of a coding agent is a property of the model *and* its harness — the code that builds context, exposes tools, caps output, and decides when work is done. Published harnesses differ by more than twenty points on the same frontier model, but that evidence is a leaderboard spread across independently engineered systems, not a controlled ablation, and it says nothing about which *component* pays. We build a harness whose seven components are independently switchable, put the grader outside the agent's reach, and run every configuration on locally served models. On a 720-episode grid (2 models $\times$ 9 configurations $\times$ 5 pinned seeds $\times$ 8 hidden-test tasks), the full bundle raises Qwen 3.8 27B from 90.0% to 95.0%, a paired Δ of $+5.0$ points whose bootstrap 95% interval $[0.0, +10.0]$ includes zero. Three findings survive the intervals. First, **the full bundle is not optimal for either model**: both peak at an ablation, not at `full`. Second, the only interval in the entire one-flag ladder that excludes zero says a component *hurt* — removing the output cap raises the weaker model by 10.0 points. Third, the capability-by-harness interaction that motivates harness engineering is **undetectable at this sample size**: all eight intervals include zero and disagree in sign. We also report two ways this measurement can silently go wrong, both of which we hit: a mid-grid protocol change inflated the same contrast to $+22.5$ points, and an API arm we tried to add was censored by the endpoint in a way that correlated with task difficulty. The artifact is 3,500 lines of dependency-free Python; every number here regenerates from one command.

1 Introduction

A coding agent is a language model plus a loop. That loop decides what the model sees on turn one, which tools it may call, how much of a tool result comes back, whether a repeated call is blocked, and whether `finish` ends the episode or is returned with failing tests. It is the harness, and changing it while holding the model fixed moves pass rate by tens of points [1, 2]. Practitioners add a system prompt, a memory file, a retry, a verifier, and then report the number as a property of the model. It is a property of a pair.

The evidence that harnesses matter is strong but coarse. On Terminal-Bench 2.0, published harnesses on a fixed base model span 13.9–37.6% for one model and 58.0–81.8% for another [1, Table 7]. That is a spread across independently engineered systems that differ in many components simultaneously. It establishes that harness choice is first-order. It cannot say which component is responsible, whether the same structure helps a 27B model on a single GPU, or whether the effect really is larger for weaker models.

The same ambiguity runs through agentic software engineering. Yang et al. [4] argue the agent–computer interface is what enables the capability. Xia et al. [5] replace the agent with a fixed pipeline and report competitive results at lower cost, suggesting much of the apparent gain was uncontrolled scaffold engineering. Zhang et al. [6] keep the loop but credit AST-aware retrieval. Each is a claim about which component pays; none is settled by holding everything else fixed and flipping one flag.

We build the instrument that makes that flip possible, and report what it measures.

Contributions.

- A coding harness with seven independently switchable components, a path-resolved sandbox, and a grader the agent cannot read, edit, or shadow (§3). 3,500 lines, Python standard library only.
- A 720-episode ablation on two local models with pinned seeds and bootstrap intervals on every cell and every paired Δ (§5), including a one-flag ladder and an explicit interaction test.

- Evidence that **the bundle is not the optimum** and that one component actively hurts the weaker model — the only ladder interval excluding zero.
- Two documented failure modes of this kind of measurement, both of which we hit and neither of which is visible in a pass rate (§6).

2 Related work

Lee et al. [1] treat harness engineering as search, with an agentic proposer rewriting harness code from prior scores and traces. Their coding-domain parent contributes the components we implement and ablate: native tool calling, a 30 kB output cap, a completion checklist [3], and an environment-bootstrap command before the first turn. We do not search over harness code; we factor one loop into flags so a single component can be switched and measured. The two compose — a proposer that rewrites a monolith cannot tell which edit paid.

Our verify gate belongs to a family that includes self-repair from execution feedback [10] and step-wise runtime verification [11]. It differs in the direction that matters for measurement: it is an *external* precondition on termination, running tests the agent cannot read or shadow, and what re-enters context is their output rather than the model's own reflection.

Benchmark validity motivates that design. Aleithan et al. [7] traced a material share of reported SWE-bench passes to solution leakage and weak tests; Lodkaew et al. [8] formalise agents reaching high scores by exploiting shortcuts rather than solving the task; Kouremetis et al. [9] document the same behaviour widely enough to inflate reported pass rates. §5.1 reports an instance caught by a check the model does not control. Finally, repeated runs of an identical prompt need not agree even with decoding fixed [13], which is why we pin a seed per repeat and report intervals rather than point estimates [14].

3 The switchable harness

The harness is a single loop around an OpenAI-compatible chat API. The model is frozen; everything deciding what it sees lives in one `Config`. Defaults are the full harness; the baseline turns every discovered component off.

Components. Seven flags, each tied to a failure observed in a local loop: `envboot` (environment snapshot before turn one), `nativetools` (native tool calls vs. shell-only), `outcap` (30 kB head-and-tail output cap), `checklist` (four questions on the first `finish`), `verifygate` (hidden tests on a later `finish`), `loopbreak` (duplicate-call detector), and `groundfs` (a read-before-write instruction). The cap keeps both ends and states how many bytes were elided, because a silently truncated log moves the evidence into the region long-context models handle worst [12].

The grader is unreachable. Three independent mechanisms, so defeating one does not defeat the others. Hidden tests never enter the sandbox. Files the task marks protected — the visible test and the written spec — are SHA-256 hashed from a pristine tree and re-checked at verification; any edit fails the episode before the hidden test runs. The hidden test is copied to a fresh temporary directory outside the sandbox and executed from there, so it cannot be shadowed by a same-named file. Every path the model supplies is resolved with `realpath` and refused outside the sandbox root, making containment a property of the harness rather than of the prompt.

Unrun is not passed. The verifier always runs after the loop stops, whatever the model claimed. A run that exhausted its context may still have fixed the code; a run that called `finish` may not have. The one deliberate asymmetry is that a wall-clock timeout scores fail even if the verifier would pass: a hung generation is not a completed task.

4 Protocol

Tasks and models. Eight hidden-test Python tasks with adversarial cases (concurrency, operational transform, NFA matching, JSON Patch, a Pratt parser, an AST transform, a stateful protocol fuzzer, distributed cache coherence). Two models served locally on one NVIDIA GH200: Qwen 3.8 27B (Q4) and Ornith-1.5 35B-A3B (Q8), $\texttt{num\_ctx} = 32768$,

Table 1: Frozen full-versus-baseline pass rate. Each cell is 40 episodes. CI is the percentile bootstrap 95% interval on the mean of five per-repeat pass rates.

Model	Config	Passes	Rate	95% CI
Qwen 3.8 27B	full	38/40	95.0%	90.0–100.0
Qwen 3.8 27B	baseline	36/40	90.0%	82.5–97.5
Ornith-1.5 35B-A3B	full	26/40	65.0%	55.0–75.0
Ornith-1.5 35B-A3B	baseline	23/40	57.5%	50.0–67.5

Table 2: One-flag-off ladder. Δ is paired full minus the named ablation; positive means the flag earned its keep. **Bold** marks the only interval excluding zero.

Config	Qwen	Qwen Δ [CI]	Ornith	Ornith Δ [CI]
full	95.0%	—	65.0%	—
baseline	90.0%	$+5.0$ $[0.0, +10.0]$	57.5%	$+7.5$ $[-5.0, +20.0]$
no-verifygate	82.5%	$+12.5$ $[-5.0, +35.0]$	62.5%	$+2.5$ $[-5.0, +10.0]$
no-envboot	95.0%	0.0 $[-10.0, +10.0]$	57.5%	$+7.5$ $[-7.5, +22.5]$
no-checklist	97.5%	-2.5 $[-10.0, +5.0]$	72.5%	-7.5 $[-20.0, +5.0]$
no-loopbreak	100%	-5.0 $[-10.0, 0.0]$	62.5%	$+2.5$ $[-15.0, +20.0]$
no-outcap	92.5%	$+2.5$ $[-7.5, +15.0]$	75.0%	$\mathbf{-10.0}$ $[\mathbf{-17.5}, \mathbf{-2.5}]$
no-groundfs	97.5%	-2.5 $[-7.5, 0.0]$	67.5%	-2.5 $[-15.0, +10.0]$
no-nativetools	97.5%	-2.5 $[-10.0, +5.0]$	55.0%	$+10.0$ $[-12.5, +32.5]$

temperature 0.6, sole tenant. An easier eleven-task suite saturated at 11/11 for the stronger model and is excluded: a model at ceiling cannot show a harness effect.

Design and statistics. $2 \times 9 \times 5 \times 8 = 720$ episodes. Repeat r pins the sampling seed to r, and the same seed is used for both arms of a paired contrast, so Δ is a paired difference of per-repeat pass rates. Cell estimates are means over five repeats; intervals are percentile bootstrap 95% CIs (10,000 resamples, RNG seed 0) [14]. With $n = 5$ the interval is the claim. The interaction statistic is $\Delta_{\text{weaker}} - \Delta_{\text{stronger}}$, with arms assigned by *empirical* full-harness mean rather than parameter count — which matters here, since the larger-parameter model is the weaker agent on this suite.

Instrument validation. Before any GPU time: 19/19 tasks verified to start broken, accept their reference solution, and reject test tampering; 106 adversarial harness tests (sandbox escapes, malformed tool calls, cap boundaries); 84 statistics tests. All green on the commit that produced the grid.

5 Results

All 720 episodes completed; every cell reports n=40 with zero starved rows.

The bundle effect is small and its interval includes zero. Qwen's five paired deltas are $+0.125, 0, +0.125, 0, 0$: never negative, but zero on three of five seeds. The mean Δ is $+5.0$ points, CI $[0.0, +10.0]$. Ornith's deltas are $+0.25, -0.125, +0.25, 0, 0$ — one seed favours the baseline — giving $+7.5$ points, CI $[-5.0, +20.0]$. Qwen's net $+2$ passes are not spread across the suite: they come from `json_patch` ($+3$) and `concurrency_race` ($+1$), against two losses on `ast_transformer`. A harness can cost a task.

The bundle is not the optimum, and one component hurts. Table 2 is the result we did not expect. *Neither model's best configuration is `full`*: Qwen peaks at `no-loopbreak` (100%, every seed on every task) and Ornith at `no-outcap` (75.0%). Removing the output cap raises the weaker model with $\Delta = -10.0$, CI $[-17.5, -2.5]$ — the only interval in the ladder excluding zero, and it says a component we shipped was costing us. The mechanism is legible: Ornith is the model living near its context ceiling, and truncating a tool result does not reduce its prompt growth

so much as remove the evidence it needed, while the truncation notice itself costs tokens. A practitioner shipping the full bundle because each component has a plausible story is shipping two components that cost their model episodes.

The interaction is undetectable. The hypothesis motivating harness engineering is that structure matters more for weaker models. Assigning arms empirically (Ornith 65.0% weaker, Qwen 95.0% stronger), all eight $\Delta_{\text{weaker}} - \Delta_{\text{stronger}}$ intervals include zero, and the point estimates do not agree in sign: four positive, three negative, one exactly zero. We report this as underpowered rather than absent. With five repeats the intervals are wide enough to be consistent with a substantial effect in either direction; a design that wants to see one needs more repeats, more tasks, or a wider capability spread than 65% versus 95%.

Cost is not wall-clock. We make no timing claim: some cells shared the GPU and later ones did not. What the transcripts support is step count — on one task the full harness finished in 14–17 steps against a baseline of 35–40, the baseline's median being the turn ceiling itself. A 43.8-hour telemetry log (78,027 samples) shows the serial batch-1 loop drawing a median 302 W against the GH200's 900 W cap while `utilization.gpu` reads 89%, with the SM clock pinned at maximum in 78,025 of 78,027 samples. Utilization near 90% here means a decode that cannot fill the device, not saturation — anyone budgeting agent-grid GPU-hours from a utilization percentage will misread what the device is doing.

5.1 A gate catches what a prompt does not

The grid ablates a bundle; one task isolates the gate against a legible failure. Its visible test checks a printed total; the hidden tests also check that a loader returns values already converted from cents, which is that module's documented contract. Without the gate the model often satisfies the visible test by breaking the contract — writing the loader to return raw cent arrays — then calls `finish` satisfied. Across matched configurations differing only in this flag, the model passes 9/9 with the gate and 5/9 without (Fisher exact $p = 0.082$); pooling adjacent single-episode tags gives 10/10 versus 7/13 ($p = 0.019$). We report both and lead with the conservative count, which does not reach $\alpha = 0.05$.

The mechanism is cleaner than the counts. Of every no-gate failure logged on this task, *all* stopped with `stop_reason = finished`: not one was a crash, a timeout, or an exhausted context. Every failure was the model asserting completion on work that did not pass. Notably the prompt-side checklist was **on in both arms** — it asks directly whether any test was weakened and whether the fix addresses the cause — and did not prevent the false finish. Only the external check did. That is a data point against prompt-level mitigation of shortcut-taking [9] and in favour of a structural gate.

6 What honest measurement costs

Two failure modes bit us. Neither is visible in a pass rate, and both would have produced a more impressive paper.

A protocol change masquerading as an effect. Our first 160 episodes used a 40-turn ceiling; we then removed it, kept already-passing episodes, and re-ran only the ceiling failures. Under that mix the same contrast reads $+22.5$ points with CI $[+10.0, +37.5]$ — *excluding zero*, four times the frozen effect, and far more publishable. It is an artifact: the baseline still contained 40-step leftovers. The tell is in the archive, where the baseline's maximum step count is exactly 40, the old ceiling, while the frozen re-run reaches 48. A 17.5-point swing in the headline came entirely from which turn ceiling the baseline had been run under.

Censoring that correlates with difficulty. We tried to add a third, API-served rung. Our first sweep produced 315 episodes with *zero* successful finishes; the cause was our own client, which sent tool results with no preceding tool-call record, so the model lost its action history every turn. After fixing that, episodes behaved normally (14–25 tool calls, clean finishes) but 32.5% of requests were refused by the endpoint under load. Those refusals were not random with respect to the outcome: one task was refused on every attempt, and both stronger models pass it 5/5. We excluded the arm. The reason is not the error rate — we keep serving errors in the denominator elsewhere, on the assumption that they are noise. Here they are not: they systematically censor the tasks requiring the most generated code. Any ablation Δ would be confounded, because flags that change how much the model writes in one call would move the refusal rate and read as a harness effect. *An error rate is reportable; an error rate correlated with difficulty is not.*

7 Limitations and conclusion

Five repeats per cell: fifteen of sixteen paired intervals include zero, and a percentile bootstrap on five points describes observed spread rather than carrying reliable coverage. Two models, both local and quantized, spanning 65–95% on one suite. The weaker arm ends 45% of its full-harness episodes in context exhaustion or serving errors, mixing capability with serving fragility. Eight Python tasks written by the same author as the harness, a bias toward failures this harness was built to catch. The design is one-flag-at-a-time, so components that only pay together, or that cancel, are invisible.

We described a coding harness small enough to run around a local 27B model and factored enough that its components can be measured rather than assumed. Turning the full bundle on raised the stronger model by 5.0 points with an interval including zero; the bundle is not optimal for either model; the one component whose interval excludes zero was *hurting* the weaker model; and the capability-by-harness interaction is undetectable at this n. The contribution we are most confident in is not an effect size. It is that a harness written as a bag of flags, with an external grader and an honest stop-reason field, converts questions usually settled by leaderboard position into questions with intervals — including the uncomfortable answer that most of them do not have one yet.

Reproducibility. Harness, tasks, runner, and statistics are dependency-free Python. Every table regenerates from one command, and figures render from the same statistics file, so a figure cannot disagree with a table. Archived runs carry provenance notes recording what they are and whether they may be used.

References

[1] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn. Meta-Harness: End-to-end optimization of model harnesses. *arXiv:2603.28052*, 2026.

[2] M. A. Merrill et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv:2601.11868*, 2026.

[3] KRAFTON AI and Ludo Robotics. Terminus-KIRA: Boosting frontier model performance on Terminal-Bench with a minimal harness. 2026. As cited by Lee et al. [1].

[4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.

[5] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv:2407.01489*, 2024.

[6] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury. AutoCodeRover: Autonomous program improvement. In *ISSTA*, 2024.

[7] R. Aleithan, H. Xue, M. M. Mohajer, E. Nnorom, G. Uddin, and S. Wang. SWE-Bench+: Enhanced coding benchmark for LLMs. *arXiv:2410.06992*, 2024.

[8] T. Lodkaew, J. Ackermann, S. Nishimori, N. Charoenphakdee, M. Sugiyama, and T. Ishida. Do coding agents deceive us? Detecting and preventing cheating via capped evaluation with randomized tests. *arXiv:2606.07379*, 2026.

[9] M. Kouremetis, A. Dawson, R. S. R. Dheekonda, and B. Greunke. Every model cheats: Prompt-level mitigation of cheating on offensive cyber tasks. *arXiv:2607.21763*, 2026.

[10] X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching large language models to self-debug. *arXiv:2304.05128*, 2023.

[11] Z. Li, Z. Wang, and J. Shang. Debug like a human: A large language model debugger via verifying runtime execution step by step. In *Findings of ACL*, 2024.

[12] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *TACL*, 12, 2024.

[13] C. Nicholson. Quantifying non-deterministic drift in large language models. *arXiv:2601.19934*, 2026.

[14] T. J. DiCiccio and B. Efron. Bootstrap confidence intervals. *Statistical Science*, 11(3):189–228, 1996.